SDLC Models

SDLC Models

- There are various software development life cycle models defined and designed which are followed during the software development process.
- These models are also referred as Software Development Process Models".
- Each process model follows a Series of steps unique to its type to ensure success in the process of software development.
- Following are the most important and popular SDLC models followed in the industry −

1)Waterfall Model

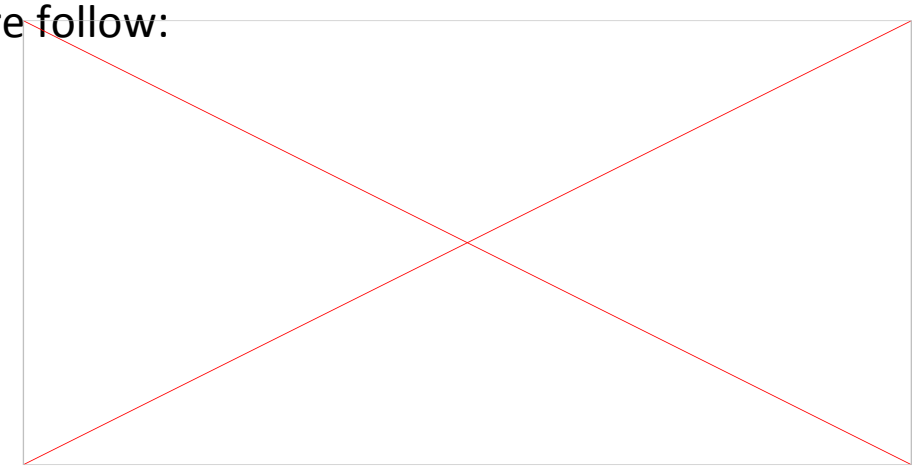
- The Waterfall Model is a Traditional Software Development Methodology.
- It was first introduced by Winston W. Royce in 1970.
- It is a linear and sequential approach to software development that consists of several phases.

Waterfall Model

- The waterfall model is a Software Development Model used in the context of large, complex projects, typically in the field of information technology.
- It is characterized by a structured, sequential approach to Project Management and Software Development.
- The Waterfall Model is useful in situations where the project requirements are well-defined and the project goals are clear.
- It is often used for large-scale projects with long timelines, where there is little room for error and the project stakeholders need to have a high level of confidence in the outcome.

Phases of Waterfall Model

- Classical Waterfall Model divides the life cycle into a set of phases. The development process can be considered as a sequential flow in the waterfall. The different sequential phases of the classical waterfall model are follow:



1.Requirements Analysis and Specification

- Requirement Analysis and specification phase aims to understand the exact requirements of the customer and document them properly. This phase consists of two different activities.
- 1. Requirement Gathering and Analysis: Firstly all the requirements regarding the software are gathered from the customer and then the gathered requirements are analyzed.
- The goal of the analysis part is to remove incompleteness (an incomplete requirement is one in which some parts of the actual requirements have been omitted) and inconsistencies (an inconsistent requirement is one in which some part of the requirement contradicts some other part).
- 2. Requirement Specification: These analyzed requirements are documented in a software requirement specification (SRS) document. SRS document serves as a contract between the development team and customers. Any future dispute between the customers and the developers can be settled by examining the SRS document.

2.Design

- The goal of this Software Design Phase is to convert the requirements acquired in the SRS into a format that can be coded in a programming language. It includes high-level and detailed design as well as the overall software architecture. A Software Design Document is used to document all of this effort (SDD).
- High-Level Design (HLD): This phase focuses on outlining the broad structure of the system. It highlights the key components and how they interact with each other, giving a clear overview of the system's architecture.
- Low-Level Design (LLD): Once the high-level design is in place, this phase zooms into the details. It breaks down each component into smaller parts and provides specifics about how each part will function, guiding the actual coding process.

3.Development

- In the Development Phase software design is translated into source code using any suitable programming language. Thus each designed module is coded. The unit testing phase aims to check whether each module is working properly or not.
- In this phase, developers begin writing the actual source code based on the designs created earlier.
- The goal is to transform the design into working code using the most suitable programming languages.
- Unit tests are often performed during this phase to make sure that each component functions correctly on its own.

4. Testing and Deployment

- Testing: Integration of different modules is undertaken soon after they have been coded and unit tested. Integration of various modules is carried out incrementally over several steps. During each integration step, previously planned modules are added to the partially integrated system and the resultant system is tested. Finally, after all the modules have been successfully integrated and tested, the full working system is obtained and system testing is carried out on this. System testing consists of three different kinds of testing activities as described below.
- Alpha testing: Alpha testing is the system testing performed by the development team.
- Beta testing: Beta testing is the system testing performed by a friendly set of customers.
- Acceptance testing: After the software has been delivered, the customer performs acceptance testing to determine whether to accept the delivered software or reject it.

Testing and Deployment

- **Deployment:** Once the software has been thoroughly tested, it's time to deploy it to the customer or end-users. This means making the software ready and available for use, often by moving it to a live or staging environment.
- During this phase, we also focus on helping users get comfortable with the software by providing training, setting up necessary environments, and ensuring everything is running smoothly. The goal is to make sure the system works as expected in real-world conditions and that users can start using it without any hitches.

5. Maintenance

- In Maintenance Phase is the most important phase of a software life cycle. The effort spent on maintenance is 60% of the total effort spent to develop a full software. There are three types of maintenance.
- Corrective Maintenance: This type of maintenance is carried out to correct errors that were not discovered during the product development phase.
- Perfective Maintenance: This type of maintenance is carried out to enhance the functionalities of the system based on the customer’s request.
- Adaptive Maintenance: Adaptive maintenance is usually required for porting the software to work in a new environment such as working on a new computer platform or with a new operating system.

Features of Waterfall Model

- **Sequential Approach:** The waterfall model involves a sequential approach to software development, where each phase of the project is completed before moving on to the next one.
- **Document-Driven:** The waterfall model depended on documentation to ensure that the project is well-defined and the project team is working towards a clear set of goals.
- **Quality Control:** The waterfall model places a high emphasis on quality control and testing at each phase of the project, to ensure that the final product meets the requirements and expectations of the stakeholders.
- **Rigorous Planning:** The waterfall model involves a careful planning process, where the project scope, timelines, and deliverables are carefully defined and monitored throughout the project lifecycle.

Importance of Waterfall Model

- Clarity and Simplicity: The linear form of the Waterfall Model offers a simple and unambiguous foundation for project development.
- Clearly Defined Phases: The Waterfall Model phases each have unique inputs and outputs, guaranteeing a planned development with obvious checkpoints.
- Documentation: A focus on thorough documentation helps with software comprehension, maintenance, and future growth.
- Stability in Requirements: Suitable for projects when the requirements are clear and stable, reducing modifications as the project progresses.
- Resource Optimization: It encourages effective task-focused work without continuously changing contexts by allocating resources according to project phases.
- Relevance for Small Projects: Economical for modest projects with simple specifications and minimal complexity.

Advantages of Waterfall Model

- Easy to Understand: The Classical Waterfall Model is very simple and easy to understand.
- Individual Processing: Phases in the Classical Waterfall model are processed one at a time.
- Properly Defined: In the classical waterfall model, each stage in the model is clearly defined.
- Clear Milestones: The classical Waterfall model has very clear and well-understood milestones.
- Properly Documented: Processes, actions, and results are very well documented.
- Reinforces Good Habits: The Classical Waterfall Model reinforces good habits like define-before-design and design-before-code.
- Working: Classical Waterfall Model works well for smaller projects and projects where requirements are well understood.

Disadvantages of Waterfall Model

- **No Feedback Path:** In the classical waterfall model evolution of software from one phase to another phase is like a waterfall. It assumes that no error is ever committed by developers during any phase. Therefore, it does not incorporate any mechanism for error correction.
- **Difficult to accommodate Change Requests:** This model assumes that all the customer requirements can be completely and correctly defined at the beginning of the project, but the customer's requirements keep on changing with time. It is difficult to accommodate any change requests after the requirements specification phase is complete.
- **No Overlapping of Phases:** This model recommends that a new phase can start only after the completion of the previous phase. But in real projects, this can't be maintained. To increase efficiency and reduce cost, phases may overlap.

Disadvantages of Waterfall Model

- **Limited Flexibility:** The Waterfall Model is a rigid and linear approach to software development, which means that it is not well-suited for projects with changing or uncertain requirements. Once a phase has been completed, it is difficult to make changes or go back to a previous phase.
- **Limited Stakeholder Involvement:** The Waterfall Model is a structured and sequential approach, which means that stakeholders are typically involved in the early phases of the project (requirements gathering and analysis) but may not be involved in the later phases (implementation, testing, and deployment).
- **Late Defect Detection:** In the Waterfall Model, testing is typically done toward the end of the development process. This means that defects may not be discovered until late in the development process, which can be expensive and time-consuming to fix.
- **Lengthy Development Cycle:** The Waterfall Model can result in a lengthy development cycle, as each phase must be completed before moving on to the next. This can result in delays and increased costs if requirements change or new issues arise.

When to Use Waterfall Model?

- Well-understood Requirements: Before beginning development, there are precise, reliable, and thoroughly documented requirements available.
- Very Little Changes Expected: During development, very little adjustments or expansions to the project's scope are anticipated.
- Small to Medium-Sized Projects: Ideal for more manageable projects with a clear development path and little complexity.
- Predictable: Projects that are predictable, low-risk, and able to be addressed early in the development life cycle are those that have known, controllable risks.
- Regulatory Compliance is Critical: Circumstances in which paperwork is of utmost importance and stringent regulatory compliance is required.
- Client Prefers a Linear and Sequential Approach: This situation describes the client's preference for a linear and sequential approach to project development.
- Limited Resources: Projects with limited resources can benefit from a set-up strategy, which enables targeted resource allocation.

Applications of Waterfall Model

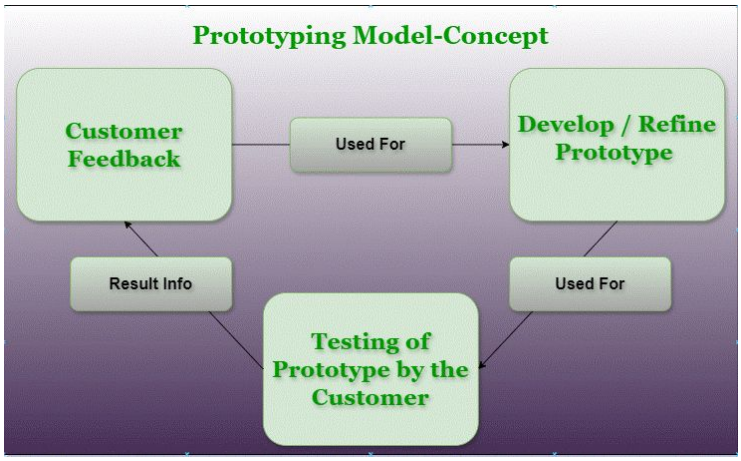
- Large-scale Software Development Projects: The Waterfall Model is often used for large-scale software development projects, where a structured and sequential approach is necessary to ensure that the project is completed on time and within budget.
- Safety-Critical Systems: The Waterfall Model is often used in the development of safety-critical systems, such as aerospace or medical systems, where the consequences of errors or defects can be severe.
- Government and Defense Projects: The Waterfall Model is also commonly used in government and defense projects, where a rigorous and structured approach is necessary to ensure that the project meets all requirements and is delivered on time.
- Projects with well-defined Requirements: The Waterfall Model is best suited for projects with well-defined requirements, as the sequential nature of the model requires a clear understanding of the project objectives and scope.
- Projects with Stable Requirements: The Waterfall Model is also well-suited for projects with stable requirements, as the linear nature of the model does not allow for changes to be made once a phase has been completed.

2)Prototyping Model

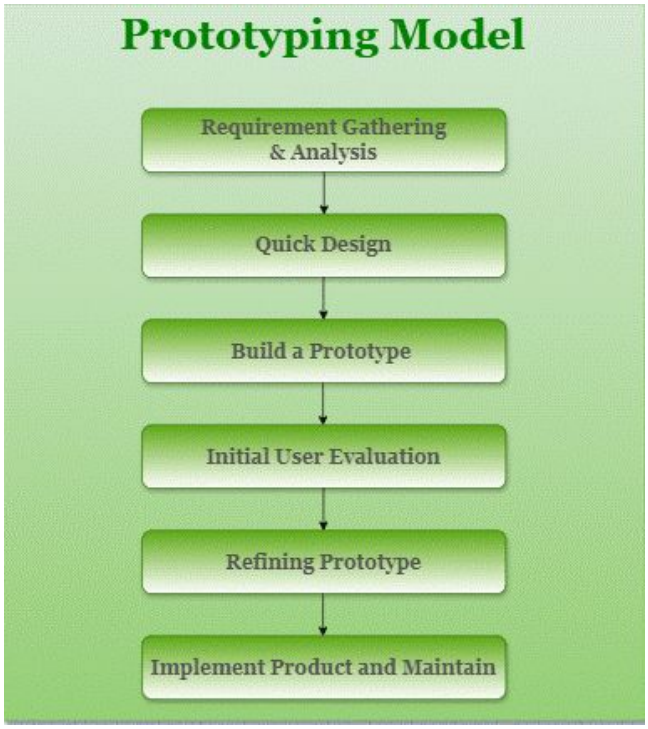
- The Prototyping Model is one of the most popularly used Software Development Life Cycle Models (SDLC models). This model is used when the customers do not know the exact project requirements beforehand.
- In this model, a prototype of the end product is first developed, tested, and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.

Prototyping Model

- In this process model, the system is partially implemented before or during the analysis phase thereby allowing the customers to see the product early in the life cycle. The process starts by interviewing the customers and developing the incomplete high-level paper model.



Prototyping Model has six phases as follows:



Prototyping Model phases:

1. Requirements gathering and analysis

- Requirement analysis is the first step in developing a prototyping model. During this phase, the system's desires are precisely defined. During the method, system users are interviewed to determine what they expect from the system.

2. Quick design

- The second phase could consist of a preliminary design or a quick design. During this stage, the system's basic design is formed. However, it is not a complete design. It provides the user with a quick overview of the system. The rapid design aids in the development of the prototype.

3. Build a Prototype

- During this stage, an actual prototype is intended to support the knowledge gained from quick design. It is a small low-level working model of the desired system.

Prototyping Model phases:

4. Initial user evaluation

- The proposed system is presented to the client for preliminary testing at this stage. It is beneficial to investigate the performance model's strengths and weaknesses. Customer feedback and suggestions are gathered and forwarded to the developer.

5. Refining prototype

- If the user is dissatisfied with the current model, you may want to improve the type that responds to user feedback and suggestions. When the user is satisfied with the upgraded model, a final system based on the approved final type is created.

6. Implement Product and Maintain

- The final system was fully tested and distributed to production after it was developed to support the original version. To reduce downtime and prevent major failures, the programmer is run on a regular basis.

Types of Prototyping Models

- 1. Rapid Throwaway Prototyping
- 2. Evolutionary Prototyping
- 3. Incremental Prototyping
- 4. Extreme Prototyping

Types of Prototyping Models

- **1. Rapid Throwaway Prototyping**
- This technique offers a useful method of exploring ideas and getting customer feedback for each of them. In this method, a developed prototype need not necessarily be a part of the accepted prototype. Customer feedback helps prevent unnecessary design faults and hence, the final prototype developed is of better quality.

Types of Prototyping Models

- **2. Evolutionary Prototyping**
- In this method, the prototype developed initially is incrementally refined based on customer feedback till it finally gets accepted. In comparison to Rapid Throwaway Prototyping, it offers a better approach that saves time as well as effort. This is because developing a prototype from scratch for every iteration of the process can sometimes be very frustrating for the developers.

Types of Prototyping Models

- **3. Incremental Prototyping**
- In this type of incremental prototyping, the final expected product is broken into different small pieces of prototypes and developed individually.
- In the end, when all individual pieces are properly developed, then the different prototypes are collectively merged into a single final product in their predefined order.
- It's a very efficient approach that reduces the complexity of the development process, where the goal is divided into sub-parts and each sub-part is developed individually.

Types of Prototyping Models

- **4. Extreme Prototyping**
- This method is mainly used for web development. It consists of three sequential independent phases:
- In this phase, a basic prototype with all the existing static pages is presented in HTML format.
- In the 2nd phase, Functional screens are made with a simulated data process using a prototype services layer.
- This is the final step where all the services are implemented and associated with the final prototype.
- This Extreme Prototyping method makes the project cycling and delivery robust and fast and keeps the entire developer team focused and centralized on product deliveries rather than discovering all possible needs and specifications and adding necessitated features.

Advantages of Prototyping Model

- The customers get to see the partial product early in the life cycle. This ensures a greater level of customer satisfaction and comfort.
- New requirements can be easily accommodated as there is scope for refinement.
- Missing functionalities can be easily figured out.
- Errors can be detected much earlier thereby saving a lot of effort and cost, besides enhancing the quality of the software.
- The developed prototype can be reused by the developer for more complicated projects in the future.
- Flexibility in design.

Advantages of Prototyping Model

- Early feedback from customers and stakeholders can help guide the development process and ensure that the final product meets their needs and expectations.
- Prototyping can be used to test and validate design decisions, allowing for adjustments to be made before significant resources are invested in development.
- Prototyping can help reduce the risk of project failure by identifying potential issues and addressing them early in the process.
- Prototyping can facilitate communication and collaboration among team members and stakeholders, improving overall project efficiency and effectiveness.
- Prototyping can help bridge the gap between technical and non-technical stakeholders by providing a tangible representation of the product.

Disadvantages of the Prototyping Model

- Costly concerning time as well as money.
- There may be too much variation in requirements each time the prototype is evaluated by the customer.
- Poor Documentation due to continuously changing customer requirements.
- It is very difficult for developers to accommodate all the changes demanded by the customer.
- There is uncertainty in determining the number of iterations that would be required before the prototype is finally accepted by the customer.
- After seeing an early prototype, the customers sometimes demand the actual product to be delivered soon.
- Developers in a hurry to build prototypes may end up with sub-optimal solutions.
- The customer might lose interest in the product if he/she is not satisfied with the initial prototype.

Disadvantages of the Prototyping Model

- The prototype may not be scalable to meet the future needs of the customer.
- The prototype may not accurately represent the final product due to limited functionality or incomplete features.
- The focus on prototype development may shift away from the final product, leading to delays in the development process.
- The prototype may give a false sense of completion, leading to the premature release of the product.
- The prototype may not consider technical feasibility and scalability issues that can arise during the final product development.
- The prototype may be developed using different tools and technologies, leading to additional training and maintenance costs.
- The prototype may not reflect the actual business requirements of the customer, leading to dissatisfaction with the final product.

Applications of Prototyping Model

- The Prototyping Model should be used when the requirements of the product are not clearly understood or are unstable.
- The prototyping model can also be used if requirements are changing quickly.
- This model can be successfully used for developing user interfaces, high-technology software-intensive systems, and systems with complex algorithms and interfaces.
- The prototyping Model is also a very good choice to demonstrate the technical feasibility of the product.